



Safety Verification of Chiron IR using Constrained Horn Clauses

Aditi Khandelia (220061)

Arush Upadhyaya (220213)

Kushagra Srivastava (220573)

Introduction

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a Property Directed Reachability (PDR)-based safety verification engine, that given a program and a set of properties on the program state, either guarantees that they hold or provides a counterexample for the same. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

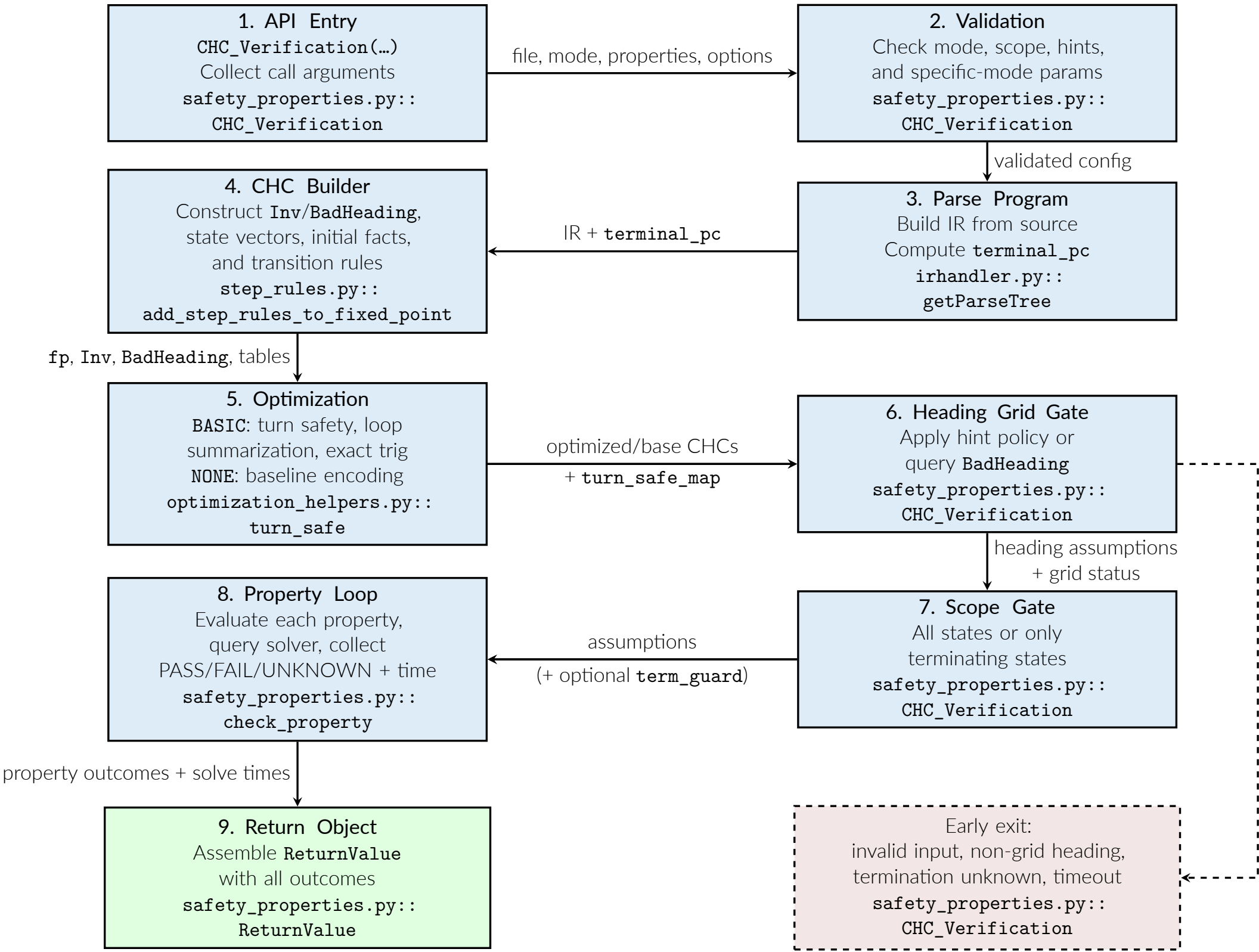
Base Flow & Tools Used

Overview of the pipeline

The pipeline takes a source turtle program and a set of safety properties as input. Using Chiron's API, the program is translated to Chiron-IR and its relevant user and state variables are extracted. The program semantics are then encoded as Constrained Horn Clauses (CHCs), and combined with the properties expressed in CHC form. This CHC system is given to Z3's SPACER engine, which checks whether an **unsafe** state is reachable. A **sat** result yields a concrete counterexample, while **unsat** proves that the property holds.

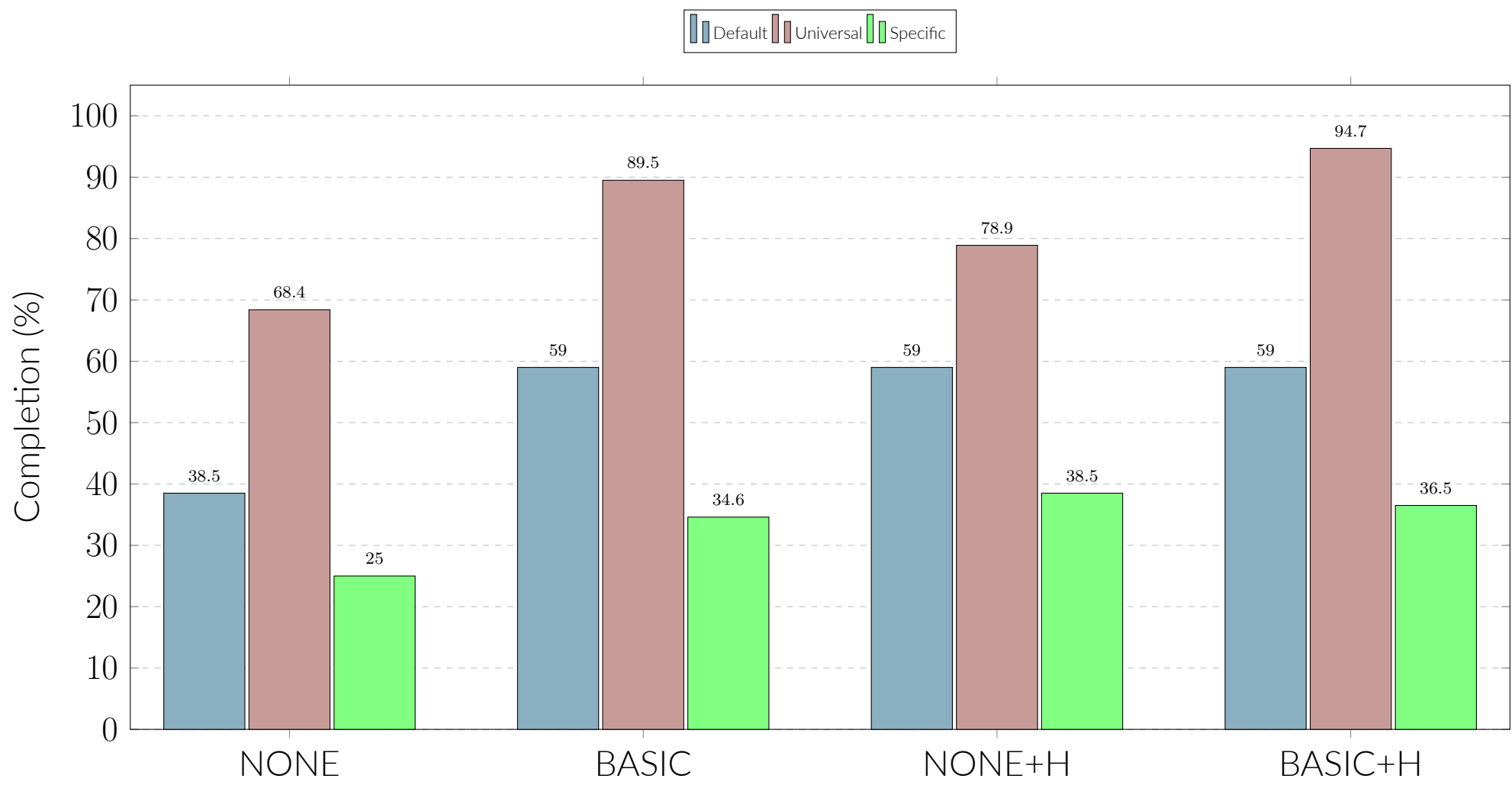
Features

Category	Option	Description
Mode	universal	Checks the property over all possible values of the user input variables, with the turtle initially at the origin and pendown=True .
	default	Checks the property with all user variables initialized to 0, with the turtle initially at the origin and pendown=False .
	specific	Similar to default , except that the user may explicitly assign values to some or all user input variables.
Property Scope	all	Interprets the safety guarantee as required to hold at every point during program execution.
	terminating	Interprets the safety guarantee as required to hold only at program termination.



Performance Analysis

Aggregate Performance



Output

Each `CHC_Verification()` call returns a verdict per property:

```
$ CHC_Verification("square.tl", "default", [...])
[PASSED] Property 'pen_up' is an invariant.
[FAILED] Property 'x_pos': counterexample found.
[UNKNOWN] Property 'heading': solver timed out.
build: 0.094 s solve: 0.03 s, 1.74 s, 60.0 s
```

- [PASSED]:** the property is an invariant.
- [FAILED]:** a counterexample was found.
- [UNKNOWN]:** the solver timed out, or returned an inconclusive result.

The output also includes the time taken for building the verification conditions, and for solving property.

Design Choices & Optimizations

Design Choices

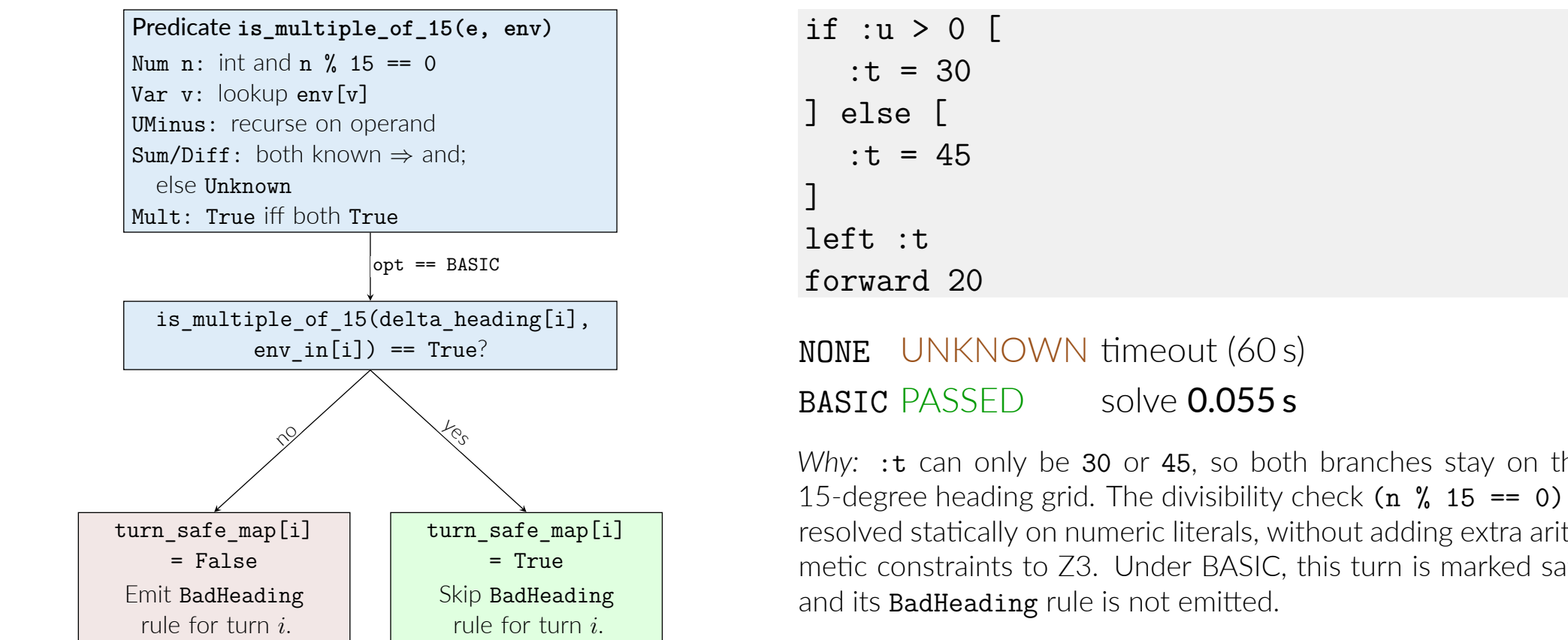
This section summarizes the principal modeling and solver choices adopted in the verifier.

- CHC semantics with Z3 Fixedpoint/SPACER:** Program behavior is encoded as CHCs and solved using Z3 Fixedpoint with `engine='spacer'` (an IC3/PDR-style Horn-clause engine), with safety posed as `Exists(s, Inv(s) & Not(P(s)))`.
- Single global invariant schema:** A unified relation `Inv(pc, xcor, ycor, heading, pendown, user_vars, loop_counters)` is used across initialization, transition rules, and property queries.
- Heading-lattice geometry model:** Heading is normalized to `[0,360)` and constrained to `{0,15,...,345}`, with violations represented explicitly via `BadHeading`.

Design Choice	Benefit	Tradeoff
CHCs solved by Z3 SPACER	Supports unbounded safety checking through inductive invariant synthesis and nonlinear arithmetic obligations; returns counterexample witnesses on solver outcomes	Performance remains sensitive to solver outcomes may be UNKNOWN violation.
Full state (pc+counters+vars)	Encodes control-state and data-state dependencies in one relation: <code>pc</code> for control/termination, <code>xcor/ycor/heading</code> for reasoning, increasing solver cost.	Properties over a strict subset of geometric predicates, and loop counters for repeat constructs.
Strict heading lattice + BadHeading	Avoids direct transcendental <code>sin/cos</code> Lookup expansion must remain constraints in SPACER by using a finite 15-degree trigonometric lookup table (precomputed in Python) with explicit yields conservative UNKNOWN <code>BadHeading</code> obligations.	Lookup expansion must remain bounded to control CHC size; failure to prove grid preservation outcomes.

Optimizations

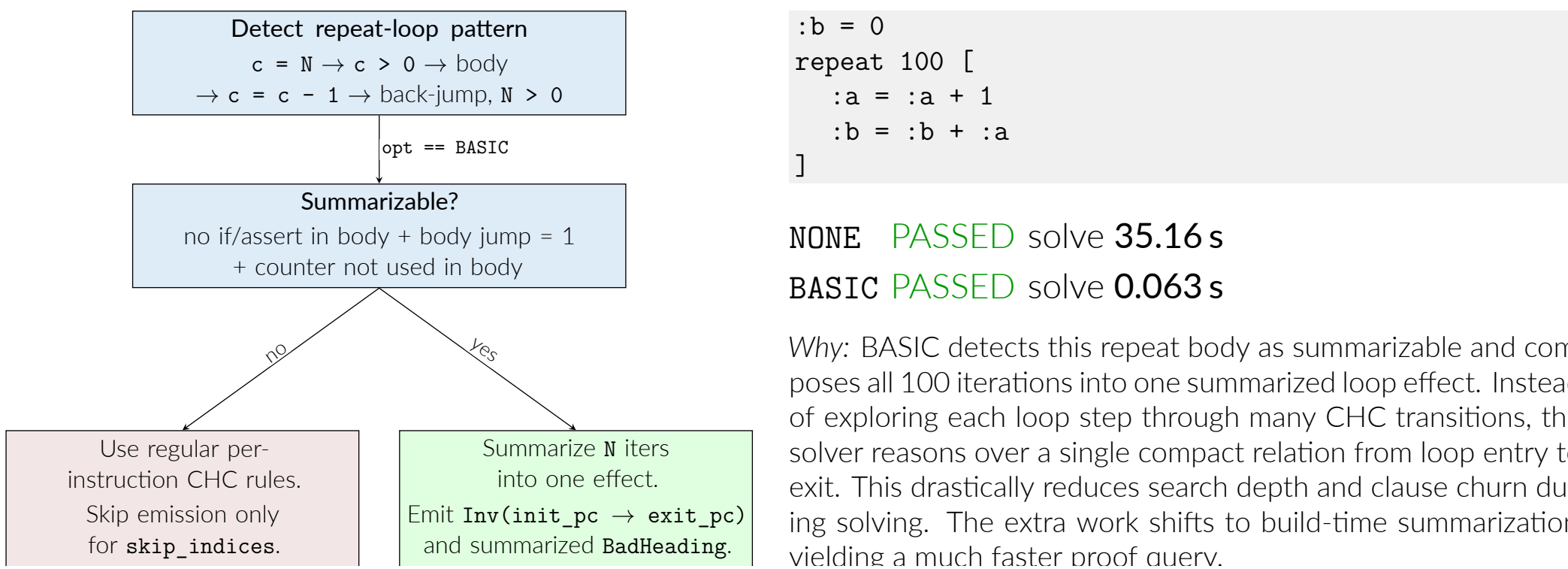
Turn-Safe Heading Inference



NONE **UNKNOWN** timeout (60 s)
BASIC **PASSED** solve 0.055 s

Why: `:t` can only be 30 or 45, so both branches stay on the 15-degree heading grid. The divisibility check (`n % 15 == 0`) is resolved statically on numeric literals, without adding extra arithmetic constraints to Z3. Under BASIC, this turn is marked safe and its `BadHeading` rule is not emitted.

Repeat-Loop Summarization



NONE **PASSED** solve 35.16 s
BASIC **PASSED** solve 0.063 s

Why: BASIC detects this repeat body as summarizable and composes all 100 iterations into one summarized loop effect. Instead of exploring each loop step through many CHC transitions, the solver reasons over a single compact relation from loop entry to exit. This drastically reduces search depth and clause churn during solving. The extra work shifts to build-time summarization, yielding a much faster proof query.

Where Optimizations Had the Biggest Impact

Case 1 : BASIC unlocks trig programs

```
# repeat 10 [ forward 10; right 90 ]
verify heading ∈ {0,90,180,270}
```

NONE **UNKNOWN** timeout (60 s)
BASIC **PASSED** solve 0.39 s

Why: NONE encodes `xcor/ycor` via `sin/cos` of a symbolic heading - Z3 cannot decide the nonlinear arithmetic. BASIC detects all turns are multiples of 15deg, switches to an integer heading grid, eliminating `sin/cos` entirely.

Case 2 : Hint trivialises heading-grid

```
# repeat 20 [ right 15; left 15 ]
verify heading ∈ 15-degree-ℤ
```

NONE **UNKNOWN** timeout (60 s)
BASIC **UNKNOWN** timeout (60 s)
NONE+H **PASSED** solve 0.107 s

Why: Both NONE and BASIC must derive heading-grid membership via CHC traversal - the 24-way disjunction always times out. The hint `heading_on_grid_always` injects it as an axiom, collapsing the proof to trivial modular arithmetic.

Case 3 : BASIC enables nested-loop inv. synth.

```
# 3-level nested loop (4³=64 iters)
verify a ≥ 0 ∧ b ≥ 0 ∧ c ≥ 0 at termination
```

NONE **UNKNOWN** timeout (3 s)
BASIC **PASSED** solve 2.29 s
BASIC+H **PASSED** solve 0.163 s

Why: NONE generates `BadHeading` CHC rules for every state, expanding the search space beyond Spacer's reach. BASIC's static turn-safety pass proves no `BadHeading` rules are needed, letting Spacer converge. Adding the hint gives a further 14x speedup.

Case 4 : BASIC cuts solve time 220× (Universal)

```
# repeat 100 [ :x = :x + 1 ]
verify x ≤ 100 at termination
```

NONE **PASSED** solve 3.54 s
BASIC **PASSED** solve 0.016 s

Why: NONE runs a heading-grid safety query first - even with no turns, Spacer traverses all reachable states, caching lemmas that hurt the main proof. BASIC proves vacuous turn-safety and skips that query entirely, keeping the solver state clean.

Future Work

- Property-aware state projection:** Project the state space onto only the variables relevant to a given property, shrinking the CHC encoding and reducing solver load.
- Stronger turn-safety check:** The current heading-grid check requires both operands of a product to be multiples of 15°. Incorporating richer program invariants will reduce unnecessary **UNKNOWN** verdicts.
- Recursive loop summarization:** For perfectly nested loops, summarize the inner loop into a single affine state transformer, then summarize the outer loop, which drastically reduces the CHC clause count.
- User-specified focus variables (focus_vars):** Let users name the variables most relevant to a property so the solver reasons over a smaller slice of the state.
- Semantic property helpers:** Replace raw property strings with domain-specific helpers such as `heading_in(range)`, `position_in_box(...)`, `pen_is_down()` for a friendlier API.
- Variable domain declarations:** Extend **specific** mode to accept intervals or sets of initial values.